

Machine Learning — Fondamentaux et pratique

Comprendre, coder et évaluer ses premiers modèles avec Scikit-learn

Contents

1	Introduction — c'est quoi le Machine Learning ?	2
1.1	Les trois grandes familles de Machine Learning	2
1.2	Régression vs classification	2
2	Scikit-learn — la bibliothèque de référence	3
2.1	Le workflow général — à retenir par cœur	3
3	Préparer les données pour un modèle	4
3.1	Encoder les variables catégorielles (texte → nombre)	4
3.2	Mettre les variables à la même échelle	4
4	Les modèles de régression — prédire un nombre	5
4.1	Régression linéaire	5
4.2	Régression polynomiale et régularisation (Ridge, Lasso)	5
5	Les modèles de classification — prédire une catégorie	6
5.1	Régression logistique	6
5.2	K plus proches voisins (KNN)	6
5.3	Arbre de décision	6
5.4	Forêt aléatoire (Random Forest)	7
6	Évaluer un modèle	8
6.1	Métriques pour la régression	8
6.2	Métriques pour la classification	8
7	Surapprentissage, sous-apprentissage et validation	9
7.1	Overfitting et underfitting	9
7.2	Validation croisée (cross-validation)	9
7.3	Recherche des meilleurs hyperparamètres	9
8	Pipeline complet et sauvegarde du modèle	10
8.1	Sauvegarder et recharger un modèle entraîné	10
9	Démarche complète d'un projet Machine Learning	11
10	Références pour aller plus loin	12

1 Introduction — c’est quoi le Machine Learning ?

Définition : le Machine Learning (apprentissage automatique) est une branche de l’intelligence artificielle où un algorithme apprend des motifs (patterns) directement à partir de données, sans qu’on lui donne des règles explicites écrites à la main. Une fois entraîné, il peut faire des prédictions sur de nouvelles données qu’il n’a jamais vues.

Pourquoi : en programmation classique, tu écris toi-même les règles (`si age < 25 alors risque_eleve`). En Machine Learning, tu donnes à l’algorithme des exemples passés (des clients, leur profil, et s’ils ont eu un sinistre ou non), et c’est lui qui découvre les règles et les relations tout seul, souvent bien plus complexes et précises que ce qu’un humain écrirait à la main.

1.1 Les trois grandes familles de Machine Learning

Type	Définition	Exemple assurance
Apprentissage supervisé	on donne à l’algorithme des exemples avec la bonne réponse déjà connue (la “cible”), il apprend à la retrouver	prédire le montant d’un sinistre, le risque d’un client
Apprentissage non supervisé	aucune réponse connue à l’avance, l’algorithme cherche des structures ou groupes cachés dans les données	segmenter les clients en profils similaires
Apprentissage par renforcement	l’algorithme apprend par essai-erreur, en recevant une récompense ou une pénalité selon ses actions	optimisation de stratégies de tarification dynamique

Dans ce cours, on se concentre sur l’**apprentissage supervisé**, le plus utilisé en data analyse et le plus adapté à ton objectif (scoring de risque, prédiction de sinistres).

1.2 Régression vs classification

À l’intérieur de l’apprentissage supervisé, il y a deux grandes questions possibles :

- **Régression :** prédire un nombre continu. *Exemple : quel sera le montant du sinistre ?* (réponse : 850,23€)
- **Classification :** prédire une catégorie. *Exemple : ce client va-t-il faire un sinistre ?* (réponse : oui/non)

Cette distinction est fondamentale : elle détermine quel modèle et quelle métrique d’évaluation utiliser tout au long du cours.

2 Scikit-learn — la bibliothèque de référence

Définition : Scikit-learn (`sklearn`) est LA bibliothèque Python de référence pour le Machine Learning classique (hors deep learning). Elle fournit des modèles prêts à l'emploi, des outils de préparation de données, et des outils d'évaluation, tous avec une syntaxe cohérente d'un modèle à l'autre.

Utilité : elle te permet de te concentrer sur la logique de ton projet (quelles données, quel modèle, comment évaluer) plutôt que de réécrire les mathématiques de chaque algorithme toi-même.

```
pip install scikit-learn
```

2.1 Le workflow général — à retenir par cœur

Peu importe le modèle utilisé, la logique Scikit-learn suit toujours les mêmes 5 étapes :

1. **Préparer les données** (X = variables explicatives, y = ce qu'on veut prédire)
2. **Séparer** en jeu d'entraînement et jeu de test (`train_test_split`)
3. **Créer et entraîner** le modèle (`.fit()`)
4. **Prédire** sur les nouvelles données (`.predict()`)
5. **Évaluer** la qualité des prédictions (comparer prédictions et vraies valeurs)

```
# Le squelette que tu retrouveras dans TOUS tes projets ML
from sklearn.model_selection import train_test_split

X = df[['age', 'nb_sinistres']] # variables explicatives (features)
y = df['prime'] # variable à prédire (target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

modele.fit(X_train, y_train) # entraînement
predictions = modele.predict(X_test) # prediction
```

Pourquoi séparer entraînement / test ?

Si tu évalues ton modèle sur les données qu'il a déjà vues pendant l'entraînement, il aura toujours l'air excellent — comme un élève qui repasserait l'examen avec les réponses déjà en main. Le jeu de test simule des données **nouvelles**, jamais vues, pour vérifier que le modèle généralise vraiment.

`test_size=0.2` : 20% des données servent de test, 80% pour l'entraînement (répartition classique, parfois 70/30). `random_state=42` : fixe le hasard du découpage pour que le résultat soit reproductible (tu peux relancer le code, tu obtiens toujours le même découpage).

3 Préparer les données pour un modèle

Définition : étape de préparation qui transforme les données nettoyées (issues de Pandas) en un format que les algorithmes de Machine Learning peuvent exploiter — exclusivement des nombres.

Utilité : un modèle ne comprend pas le texte ("auto", "habitation") ni les échelles très différentes entre colonnes (âge de 18 à 75, prime de 100 à 2000) — il faut tout convertir et harmoniser.

3.1 Encoder les variables catégorielles (texte → nombre)

```
# One-Hot Encoding : cree une colonne binaire (0/1) par categorie
df_encode = pd.get_dummies(df, columns=['type_contrat', 'region'])

# LabelEncoder : transforme chaque categorie en un entier (0, 1, 2...)
from sklearn.preprocessing import LabelEncoder
encodeur = LabelEncoder()
df['type_contrat_code'] = encodeur.fit_transform(df['type_contrat'])
```

Quand utiliser lequel ? Le One-Hot Encoding est préférable pour la plupart des modèles (régression, réseaux de neurones) car il n'introduit pas d'ordre artificiel entre les catégories. Le LabelEncoder convient surtout aux arbres de décision et forêts aléatoires, qui ne sont pas perturbés par un ordre arbitraire.

3.2 Mettre les variables à la même échelle

Définition : technique qui ramène toutes les variables numériques sur une échelle comparable (par exemple entre 0 et 1, ou centrée-réduite).

Utilité : certains modèles (régression linéaire, KNN, SVM) sont sensibles à l'échelle des variables : une colonne allant de 0 à 100 000 (revenu) écraserait une colonne allant de 0 à 1 (ratio) dans le calcul, même si les deux sont aussi importantes.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# StandardScaler : centre-reduit (moyenne=0, ecart-type=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # attention : transform seul, pas
fit_transform

# MinMaxScaler : ramene toutes les valeurs entre 0 et 1
scaler2 = MinMaxScaler()
X_train_norm = scaler2.fit_transform(X_train)
```

Règle essentielle : on calcule les paramètres de mise à l'échelle (**fit**) uniquement sur le jeu d'entraînement, puis on les applique (**transform**) au jeu de test — jamais l'inverse, sinon le modèle "voit" indirectement des informations du jeu de test avant l'évaluation (fuite de données / data leakage).

4 Les modèles de régression — prédire un nombre

4.1 Régression linéaire

Définition : modèle qui cherche la meilleure ligne droite (ou plan, en plusieurs dimensions) reliant les variables explicatives à la variable cible, en minimisant l'écart entre les prédictions et les vraies valeurs.

Utilité : premier modèle à essayer sur un problème de régression : simple, rapide, et interprétable — on peut lire directement l'effet de chaque variable.

```
from sklearn.linear_model import LinearRegression

modele = LinearRegression()
modele.fit(X_train, y_train)
predictions = modele.predict(X_test)

# Interpreter le modele
print(modele.coef_) # poids attribue a chaque variable
print(modele.intercept_) # valeur de base (quand toutes les variables valent 0)
```

Chaque coefficient (`coef_`) indique de combien la prédiction change quand la variable associée augmente de 1 unité, toutes les autres variables restant fixes.

4.2 Régression polynomiale et régularisation (Ridge, Lasso)

Définition : extensions de la régression linéaire : la régression polynomiale capture des relations non linéaires (courbes), tandis que Ridge et Lasso ajoutent une pénalité pour éviter que le modèle ne devienne trop complexe.

Utilité : utile quand la relation entre variables n'est pas une simple droite, ou quand il y a beaucoup de variables et un risque de surapprentissage.

```
from sklearn.linear_model import Ridge, Lasso

modele_ridge = Ridge(alpha=1.0) # penalise les gros coefficients
modele_lasso = Lasso(alpha=0.1) # peut ramener certains coefficients a 0 (selection de variables)
modele_ridge.fit(X_train, y_train)
```

5 Les modèles de classification — prédire une catégorie

5.1 Régression logistique

Définition : malgré son nom, c'est un modèle de **classification**, pas de régression. Il calcule une probabilité (entre 0 et 1) qu'une observation appartienne à une catégorie, via la fonction sigmoïde.

Utilité : premier modèle à essayer sur un problème de classification binaire (oui/non) — simple et rapide, comme la régression linéaire pour la régression.

```
from sklearn.linear_model import LogisticRegression

modele = LogisticRegression()
modele.fit(X_train, y_train_classe)
predictions = modele.predict(X_test) # classe prédite (0 ou 1)
probabilites = modele.predict_proba(X_test) # probabilité de chaque classe
```

5.2 K plus proches voisins (KNN)

Définition : pour classer une nouvelle observation, l'algorithme regarde ses K voisins les plus proches (selon les données d'entraînement) et lui attribue la catégorie majoritaire parmi eux.

Utilité : intuitif et efficace sur de petits datasets ; nécessite obligatoirement une mise à l'échelle des variables (StandardScaler) car il se base sur des distances.

```
from sklearn.neighbors import KNeighborsClassifier

modele = KNeighborsClassifier(n_neighbors=5)
modele.fit(X_train_scaled, y_train)
```

5.3 Arbre de décision

Définition : modèle qui découpe successivement les données par une série de questions simples ("age < 25 ?", "nb_sinistres > 1 ?"), formant un arbre de décisions jusqu'à une prédiction finale.

Utilité : facilement interprétable — on peut littéralement dessiner et lire les règles de décision, ce qui est précieux en assurance pour justifier un refus ou un tarif à un client ou un régulateur.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

modele = DecisionTreeClassifier(max_depth=4, random_state=42)
modele.fit(X_train, y_train)

plt.figure(figsize=(15, 8))
plot_tree(modele, feature_names=X.columns, filled=True)
plt.show()
```

`max_depth` : profondeur maximale de l'arbre — limite essentielle pour éviter qu'il ne mémorise les données par cœur (surapprentissage).

5.4 Forêt aléatoire (Random Forest)

Définition : combine un grand nombre d'arbres de décision différents (chacun entraîné sur un sous-échantillon aléatoire des données), puis fait voter (classification) ou moyenne (régression) leurs prédictions.

Utilité : généralement plus précis et plus stable qu'un seul arbre, au prix d'une interprétabilité un peu réduite — souvent le meilleur compromis performance/simplicité pour démarrer un vrai projet.

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

modele = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
modele.fit(X_train, y_train)

# Importance de chaque variable dans les decisions du modele
importances = pd.Series(modele.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False))
```

`n_estimators` : nombre d'arbres dans la forêt (plus il y en a, plus stable, mais plus lent).
`feature_importances_` : indique quelles variables ont le plus influencé les décisions du modèle — très utile pour expliquer un résultat.

6 Évaluer un modèle

6.1 Métriques pour la régression

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, predictions) # erreur absolue moyenne
mse = mean_squared_error(y_test, predictions) # erreur quadratique moyenne
rmse = np.sqrt(mse) # meme unite que y (plus lisible)
r2 = r2_score(y_test, predictions) # part de variance expliquée
```

Métrique	Interprétation
MAE	erreur moyenne en valeur absolue, dans l'unité d'origine (ex : en euros) — facile à expliquer
RMSE	similaire au MAE mais pénalise davantage les grosses erreurs — plus sensible aux outliers
R ²	entre 0 et 1 (peut être négatif si le modèle est très mauvais) — proche de 1 = le modèle explique bien la variation des données

6.2 Métriques pour la classification

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)

# Vue d'ensemble en une commande
print(classification_report(y_test, predictions))

# Matrice de confusion : le detail des bonnes/mauvaises predictions
print(confusion_matrix(y_test, predictions))
```

Métrique	Interprétation
Accuracy	% total de prédictions correctes — trompeur si les classes sont déséquilibrées (ex : 95% de clients sans sinistre)
Precision	parmi les clients prédits “à risque”, combien le sont vraiment — important quand une fausse alerte coûte cher
Recall	parmi les clients réellement “à risque”, combien ont été détectés — important quand rater un cas est grave (ex : fraude)
F1-score	moyenne équilibrée entre precision et recall — bon indicateur global quand les deux comptent

Piège classique : sur un dataset où 95% des clients n'ont pas de sinistre, un modèle qui prédit "pas de sinistre" à chaque fois obtient déjà 95% d'accuracy sans rien avoir appris. C'est pour cela qu'il faut toujours regarder precision/recall/F1 en plus de l'accuracy sur des classes déséquilibrées.

7 Surapprentissage, sous-apprentissage et validation

7.1 Overfitting et underfitting

Problème	Définition	Signe
Overfitting (surapprentissage)	le modèle mémorise les données d'entraînement par cœur, y compris leur bruit, et généralise mal	score excellent en entraînement, mauvais en test
Underfitting (sous-apprentissage)	le modèle est trop simple pour capturer les relations réelles des données	score médiocre en entraînement ET en test

```
score_train = modele.score(X_train, y_train)
score_test = modele.score(X_test, y_test)
print(f"Train : {score_train:.3f} | Test : {score_test:.3f}")
# Un grand écart (train >> test) = signe d'overfitting
```

7.2 Validation croisée (cross-validation)

Définition : au lieu d'un seul découpage train/test, on découpe les données en K parts (folds) ; le modèle est entraîné K fois, à chaque fois sur K-1 parts et testé sur la part restante, puis on moyenne les résultats.

Utilité : donne une estimation plus fiable et plus stable de la performance réelle du modèle qu'un simple découpage unique, qui pourrait être chanceux ou malchanceux.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modele, X, y, cv=5) # 5 découpages différents
print(f"Score moyen : {scores.mean():.3f} (+/- {scores.std():.3f})")
```

7.3 Recherche des meilleurs hyperparamètres

Définition : les hyperparamètres sont les réglages d'un modèle choisis avant l'entraînement (ex : `max_depth`, `n_estimators`) — `GridSearchCV` teste automatiquement plusieurs combinaisons et garde la meilleure, évaluée par validation croisée.

```
from sklearn.model_selection import GridSearchCV

parametres = {
    'max_depth': [3, 5, 7, 10],
    'n_estimators': [50, 100, 200]
}

recherche = GridSearchCV(RandomForestClassifier(random_state=42),
                          parametres, cv=5, scoring='f1')
recherche.fit(X_train, y_train)

print(recherche.best_params_) # meilleure combinaison trouvee
meilleur_modele = recherche.best_estimator_
```

8 Pipeline complet et sauvegarde du modèle

Définition : un Pipeline enchaîne automatiquement plusieurs étapes (mise à l'échelle, encodage, modèle) en un seul objet, appliqué de façon cohérente à l'entraînement et à la prédiction.

Utilité : évite d'oublier une étape de préparation lors de la prédiction sur de nouvelles données, et rend le code plus propre et réutilisable.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('modele', RandomForestClassifier(n_estimators=100, random_state=42))
])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

8.1 Sauvegarder et recharger un modèle entraîné

```
import joblib

joblib.dump(pipeline, 'modele_risque_assurance.pkl') # sauvegarde

modele_charge = joblib.load('modele_risque_assurance.pkl') # rechargement
modele_charge.predict(nouvelles_donnees)
```

9 Démarche complète d'un projet Machine Learning

1. **Définir le problème** : régression ou classification ? quelle variable cible ?
2. **Charger et explorer** les données (Pandas, `head`, `info`, `describe`)
3. **Nettoyer** : valeurs manquantes, doublons, incohérences
4. **Analyser visuellement** (Matplotlib/Seaborn) pour comprendre les relations
5. **Préparer** : encodage des catégories, mise à l'échelle
6. **Séparer** en train/test
7. **Entraîner** un modèle simple d'abord (régression linéaire ou logistique) comme référence (baseline)
8. **Évaluer** avec les métriques adaptées
9. **Améliorer** : essayer des modèles plus complexes (forêt aléatoire), optimiser les hyperparamètres
10. **Interpréter** : quelles variables comptent le plus ? le modèle est-il fiable et explicable ?
11. **Sauvegarder** et documenter le modèle final

Conseil : toujours commencer par un modèle simple (baseline) avant d'aller vers des modèles complexes. Si la régression logistique obtient déjà 80% de F1-score, ça donne un point de comparaison clair pour juger si une forêt aléatoire apporte vraiment un gain, ou complique inutilement les choses.

10 Références pour aller plus loin

Références académiques et documentaires de référence, utiles pour approfondir chaque notion abordée dans ce cours.

- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: with Applications in R*. 2nd edn. New York: Springer. Disponible gratuitement à : <https://www.statlearning.com>
- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edn. New York: Springer. Disponible gratuitement à : <https://hastie.su.domains/ElemStatLearn/>
- Géron, A. (2022) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd edn. Sebastopol: O'Reilly Media.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Disponible à : <https://jmlr.org/papers/v12/pedregosa11a.html>
- Scikit-learn developers (2024) *Scikit-learn User Guide*. Disponible à : https://scikit-learn.org/stable/user_guide.html
- Harvard University (2024) *CS109: Introduction to Data Science*. Harvard John A. Paulson School of Engineering and Applied Sciences. Disponible à : <https://cs109.github.io/>
- Harvard University, Data Science Series (2024) *Data Science: Machine Learning (edX/HarvardX PH125.8x)*. Disponible à : <https://www.edx.org/learn/machine-learning>